



# Complete Detailed Tutorial on Linux File Permissions

Linux is a **multi-user operating system**, so permissions are very important for security and access control.

Every file and directory in Linux has:

-  **Owner (User)**
  -  **Group**
  -  **Others**
  -  **Permissions (Read, Write, Execute)**
- 

## 1 Understanding Permission Basics

Run:

```
ls -l
```

Example output:

```
-rwxr-xr-- 1 ashvi devops 1200 Feb 16 file.txt
```

Let's break this down:

```
-rwxr-xr--
|| || || |
|| || |  └─ Others (r--)
|| |    └─ Group (r-x)
|| └──   └─ Owner (rwx)
| └───── └─ File type
```

\_\_\_\_\_ Permission string

---

## 2 File Types (First Character)

| Symbol | Meaning          |
|--------|------------------|
| -      | Regular file     |
| d      | Directory        |
| l      | Symbolic link    |
| c      | Character device |
| b      | Block device     |

Example:

```
drwxr-xr-x  → Directory
-rw-r--r--  → File
```

---

## 3 Permission Types

Each user category has 3 permissions:

| Symbol | Name    | Meaning                    |
|--------|---------|----------------------------|
| r      | Read    | View content               |
| w      | Write   | Modify content             |
| x      | Execute | Run file / enter directory |

---

# 4 Permission Meaning for Files vs Directories

## For Files

| Permission | Meaning          |
|------------|------------------|
| r          | Can read file    |
| w          | Can modify file  |
| x          | Can execute file |

Example:

`-rwxr-xr--`

Owner can read/write/execute.  
Group can read & execute.  
Others can only read.

---

## For Directories

| Permission | Meaning             |
|------------|---------------------|
| r          | List files          |
| w          | Create/Delete files |
| x          | Enter directory     |

Example:

`drwxr-x---`

Owner can:

- List
- Enter
- Create/Delete files

Group can:

- List
- Enter

Others:

- No access
- 

## **5** User Categories

Linux divides users into 3 categories:

### **1. Owner (u)**

Person who created the file.

### **2. Group (g)**

Users in the same group.

### **3. Others (o)**

All other users.

---

## **6** Changing Permissions – chmod

There are 2 methods:

1. Symbolic Method
  2. Numeric (Octal) Method
- 

## ◆ Method 1: Symbolic Method

Syntax:

```
chmod [who][operator][permission] file
```

**Who:**

- u → user
- g → group
- o → others
- a → all

**Operator:**

- + → Add
  - - → Remove
  - = → Assign exact permission
- 

## Examples

**Give execute permission to owner**

```
chmod u+x file.txt
```

**Remove write from group**

```
chmod g-w file.txt
```

### Give read to others

```
chmod o+r file.txt
```

### Give execute to all

```
chmod a+x file.txt
```

### Set exact permission

```
chmod u=rwx,g=rx,o=r file.txt
```

---

## ◆ Method 2: Numeric (Octal) Method

Each permission has a number:

| Permission | Value |
|------------|-------|
|------------|-------|

|   |   |
|---|---|
| r | 4 |
|---|---|

|   |   |
|---|---|
| w | 2 |
|---|---|

|   |   |
|---|---|
| x | 1 |
|---|---|

Add values together.

Examples:

| Permission | Calculation | Value |
|------------|-------------|-------|
|------------|-------------|-------|

|     |       |   |
|-----|-------|---|
| rwx | 4+2+1 | 7 |
|-----|-------|---|

|     |     |   |
|-----|-----|---|
| rw- | 4+2 | 6 |
|-----|-----|---|

|     |     |   |
|-----|-----|---|
| r-x | 4+1 | 5 |
|-----|-----|---|

r--            4            4

---

## Example:

```
chmod 755 file.txt
```

Means:

- 7 → Owner (rwx)
  - 5 → Group (r-x)
  - 5 → Others (r-x)
- 

## Common Permission Numbers

| Number | Meaning                           |
|--------|-----------------------------------|
| 777    | Full access to everyone           |
| 755    | Owner full, others read & execute |
| 700    | Only owner full access            |
| 644    | Owner read/write, others read     |
| 600    | Owner read/write only             |

---

## **7** Changing Ownership – chown

Syntax:

```
chown user file
```

Change owner:

```
chown ashvi file.txt
```

Change owner & group:

```
chown ashvi:devops file.txt
```

Recursive:

```
chown -R ashvi:devops directory/
```

---

## 8 Changing Group – chgrp

```
chgrp devops file.txt
```

---

## 9 Recursive Permission Change

Apply permission to directory and its contents:

```
chmod -R 755 project/
```

 Be careful with recursive permission changes.

---

## 10 Special Permissions

There are 3 special permissions:

1. SUID
2. SGID
3. Sticky Bit



---

# SUID (Set User ID)

## Definition

SUID is a special permission bit set on an **executable file**.

When enabled:

The program runs with the **effective UID of the file owner**, not the user executing it.

---

## How It Looks

-rwsr-xr-x

^

s → SUID bit

Numeric form:

chmod 4755 filename

4 → SUID bit

---

## Important Concept: Real vs Effective UID

| Term          | Meaning                         |
|---------------|---------------------------------|
| Real UID      | User who started the process    |
| Effective UID | User whose permissions are used |

👉 SUID changes **Effective UID** only.

---

## Practical Verification Using C Program

## Step 1: Program to Check UID

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Real UID: %d\n", getuid());
    printf("Effective UID: %d\n", geteuid());
    return 0;
}
```

Compile:

```
gcc checkuid.c -o checkuid
```

---

## Step 2: Enable SUID

```
sudo chown root:root checkuid
sudo chmod 4755 checkuid
```

Verify:

```
ls -l checkuid
```

Output:

```
-rwsr-xr-x 1 root root 16384 checkuid
```

---

## Step 3: Run as Normal User

```
./checkuid
```

Example Output:

```
Real UID: 1001
Effective UID: 0
```

### ✓ Proof:

- Real UID → your user
- Effective UID → root (0)
- Program is executing with root privileges

That confirms SUID is working.

---

## ✓ Real System Example

`/usr/bin/passwd`

`ls -l /usr/bin/passwd`

Shows:

`-rwsr-xr-x 1 root root ...`

It must run as root to modify `/etc/shadow`.

---

## ⚠ Security Risk

If a program with SUID:

- Executes system commands
- Has buffer overflow
- Is poorly written

It can lead to **privilege escalation**.

---

## 12 👤 SGID (Set Group ID)

### 📌 Definition

SGID works in two ways:

1. On executables → runs with file's group

2. On directories → new files inherit directory's group

---

## How It Looks

`-rwxr-sr-x`

^

s → SGID

Numeric form:

`chmod 2755 filename`

2 → SGID bit

---

## Practical Verification (Executable)

### Step 1: C Program to Check GID

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {  
  
    printf("Real GID: %d\n", getgid());  
  
    printf("Effective GID: %d\n", getegid());  
  
    return 0;  
}
```

Compile:

```
gcc checkgid.c -o checkgid
```

---

## Step 2: Enable SGID

```
sudo chown root:devteam checkgid
```

```
sudo chmod 2755 checkgid
```

---

## Step 3: Run as Normal User

```
./checkgid
```

Example Output:

Real GID: 1001

Effective GID: 1005

✓ Effective GID changes to file's group → SGID confirmed.

---

# SGID on Directory (Very Common in Real World)

Create shared directory:

```
sudo mkdir /project
```

```
sudo chown root:devteam /project
```

```
sudo chmod 2775 /project
```

Check:

```
ls -ld /project
```

Output:

```
drwxrwsr-x
```

Now create file inside:

```
touch /project/test.txt
```

```
ls -l /project/test.txt
```

Output:

```
-rw-r--r-- 1 user devteam test.txt
```

✓ Group automatically inherited → SGID working.

---

## Interview-Ready Summary

| Feature     | SUID             | SGID                      |
|-------------|------------------|---------------------------|
| Affects     | User             | Group                     |
| Changes     | Effective UID    | Effective GID             |
| Used For    | Privileged tasks | Shared access             |
| Works On    | Executables      | Executables & Directories |
| Numeric Bit | 4                | 2                         |

---

## Sticky Bit

### ◆ Definition

The **Sticky Bit** is a special permission bit set **only on directories**.

When enabled:

Only the **file owner**, **directory owner**, or **root** can delete or rename files inside that directory — even if others have write permission.

---

## How It Looks

`drwxrwxrwt`

^

t → Sticky Bit

Numeric form:

`chmod 1777 directory`

1 → Sticky Bit

---

## Why Sticky Bit Exists

Imagine a directory with:

`chmod 777 shared_folder`

Anyone can:

- Create files
- Modify files
-  Delete other users' files

Sticky Bit solves the delete problem.

---

## Real System Example

## /tmp Directory

Check it:

```
ls -ld /tmp
```

Typical output:

```
drwxrwxrwt 10 root root 4096 Mar  3 /tmp
```

- Writable by everyone (777)
- Protected by sticky bit (t)

Without sticky bit:

- Any user could delete another user's temporary files

---

## Practical Example (Hands-On)

### Step 1: Create Test Directory

```
mkdir shared
```

```
chmod 1777 shared
```

```
ls -ld shared
```

Output:

```
drwxrwxrwt
```

Sticky bit confirmed.

---

### Step 2: Create Files as Different Users

User1:



```
touch shared/user1_file
```

User2:

```
rm shared/user1_file
```

### Result:

rm: cannot remove 'shared/user1\_file': Operation not permitted

✓ Deletion blocked.

---

### t vs T Difference

| Symbol | Meaning                              |
|--------|--------------------------------------|
| t      | Sticky bit + execute permission      |
| T      | Sticky bit but no execute permission |

Example:

```
chmod 1766 shared
```

```
ls -ld shared
```

Output:

```
drwxrw-rwT
```

Uppercase T means execute is missing for others.

---

### Important Rule

Sticky bit affects **only deletion & renaming**, not:

- Reading
  - Writing
  - Executing
- 

## Security Use Case

Common in:

- `/tmp`
- `/var/tmp`
- Shared hosting environments
- Multi-user systems

Prevents:

- Accidental deletion
  - Malicious deletion
- 

## Permission Summary (All Three Special Bits)

| Bit  | Number | Works On    | Changes       |
|------|--------|-------------|---------------|
| SUID | 4      | Executables | Effective UID |

|        |   |                           |                                   |
|--------|---|---------------------------|-----------------------------------|
| SGID   | 2 | Executables & Directories | Effective GID / Group Inheritance |
| Sticky | 1 | Directories               | Delete protection                 |

---

### Key Rule:

- To delete a file in Linux, you need **write (w)** and **execute (x)** permission on the **containing directory, not on the file itself**.
  - The file's permissions control **reading/writing content**, not deletion.
  - The directory's permissions control **creating, renaming, or deleting files** inside it.
- 

## 14 Understanding 3-Digit & 4-Digit Permission

Example:

```
chmod 1777 dir
```

1 → Sticky bit

777 → rwxrwxrwx

4-digit format:

```
Special Owner Group Others
```

Values:

- 4 → SUID
- 2 → SGID
- 1 → Sticky

---

## Default Permission & umask

When file is created:

Default:

- File → 666
- Directory → 777

But the system subtracts **umask** value.

Check:

`umask`

Example:

`0022`

New file permission:

`666 - 022 = 644`

New directory:

`777 - 022 = 755`

---

## What is `umask` in Linux?

`umask` stands for:

**User File Creation Mode Mask**

In simple words:

👉 **umask** decides what permissions should be **REMOVED** from new files and directories.

It does NOT add permissions.

It only **removes (masks)** permissions.

---



## Simple Real-Life Example

Imagine:

- Every new file is created with **maximum open access**
- But you don't want everyone to modify it
- So you set a rule to remove some permissions

That rule = **umask**

---



## Default Permissions Before umask

When Linux creates something:

| Type      | Default Permission |
|-----------|--------------------|
| File      | 666 (rw-rw-rw-)    |
| Directory | 777 (rwxrwxrwx)    |

⚠ Files don't get execute by default.

---



## How umask Works

Let's say:

umask 0022

Now we subtract it.

---

## For Files

Default = 666

Umask = 022

Calculation:

666

-022

----

644

Result:

rw-r--r--

So new files become:

-rw-r--r--

---

## For Directories

Default = 777

Umask = 022

Calculation:

777

-022

----

755

Result:

rwxr-xr-x

---



# What Does 0022 Mean?

Break it:

0 0 2 2

| Position | Meaning                               |
|----------|---------------------------------------|
| 1st 0    | Special permissions (ignored usually) |
| 2nd 0    | Owner (remove nothing)                |
| 3rd 2    | Remove write from group               |
| 4th 2    | Remove write from others              |

So:

- Owner → full access
- Group → read & execute only
- Others → read & execute only



## Summary Table

| File                         | Scope  | When Applied      |
|------------------------------|--------|-------------------|
| <code>/etc/login.defs</code> | System | Login session     |
| <code>/etc/profile</code>    | System | Login shell       |
| <code>/etc/bashrc</code>     | System | Interactive shell |

|                         |      |                   |
|-------------------------|------|-------------------|
| <code>~/.profile</code> | User | Login shell       |
| <code>~/.bashrc</code>  | User | Interactive shell |

---

## Quick Summary

| Concept | Meaning                    |
|---------|----------------------------|
| r       | Read                       |
| w       | Write                      |
| x       | Execute                    |
| chmod   | Change permission          |
| chown   | Change owner               |
| chgrp   | Change group               |
| SUID    | Run as owner               |
| SGID    | Run as group               |
| Sticky  | Protect shared directory   |
| umask   | Default permission control |

---